

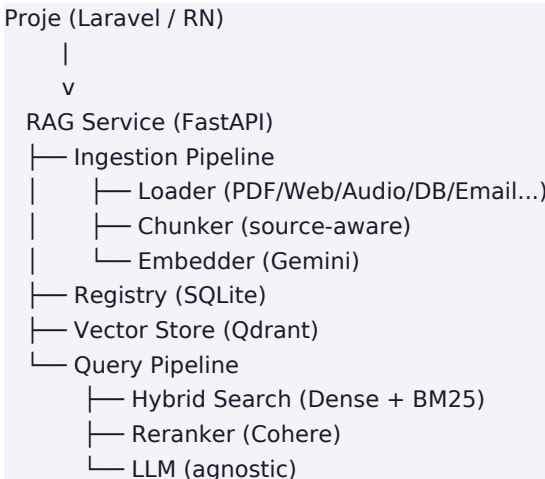
RAG Service

Bağımsız, Çok Kaynaklı Retrieval-Augmented Generation Servisi

Mimari Karar Belgesi — v1.0

1. Genel Mimari

Servis, her projeden HTTP ile çağrılabilen, bağımsız bir FastAPI uygulamasıdır. Ingestion ve query path'leri birbirinden ayrıdır. Tüm projeler aynı servis instance'ını kullanır; izolasyon project_id ile sağlanır.



2. Temel Teknoloji Kararları

2.1 Vector Store — Qdrant

Qdrant, Docker container olarak deploy edilir. Güçlü payload filtering ve native hybrid search desteği sunar. Her proje project_id payload alanıyla izole edilir; ayrı collection açılmaz.

2.2 Metadata / Registry — SQLite

Ingestion yönetimi için SQLite yeterlidir; dışarıya bağımlılık yoktur. İki temel tablo:

Tablo	İçerik
rag_documents	source_type, source_ref, content_hash, status, embedded_at
rag_chunks	document_id, qdrant_point_id, chunk_index, token_count

→ qdrant_point_id, SQLite ile Qdrant arasındaki köprüdür. Re-index ve silme işlemlerinde kullanılır.

2.3 Embedding — Gemini

Model	Dimension	Kullanım
gemini-embedding-exp-03-07	768 (truncated)	Primary — yüksek kalite, Türkçe güçlü
text-embedding-004	768	Fallback — stabil, rate limit yoksa

→ Her zaman 768 dimension kullan. Primary model 3072 üretir, truncate edilir. Böylece Qdrant collection sabit kalır, model değişince yeniden index gerekmez.

→ task_type: index ederken RETRIEVAL_DOCUMENT, query ederken RETRIEVAL_QUERY.

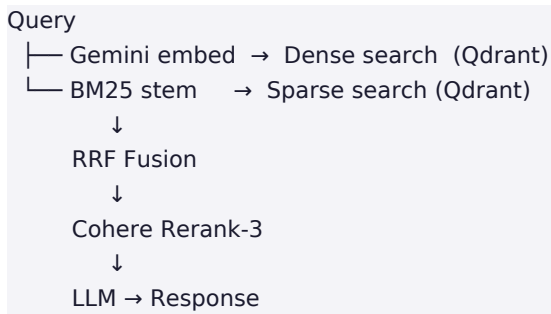
2.4 Chunking Stratejisi

Source	Strateji	Parent	Child	Overlap
PDF	Semantic (paragraf sonu)	1500 token	300 token	50 token
Web	Semantic (HTML temizlenmiş)	1000 token	250 token	50 token
Audio	Speaker turn bazlı	Konuşma bloğu	500 token (max)	Yok
DB	Record bazlı (flatten)	Row	Row	Yok
Email/Chat	Thread → mesaj bazlı	Thread	Mesaj	Yok

→ Doğal sınır varsa (speaker turn, paragraf, mesaj) ona göre böl. Her chunk metadata'sında pozisyon bilgisi olmalı.

2.5 Retrieval — Hybrid Search

Dense (semantic) + Sparse (BM25) arama yapılır, RRF ile birleştirilir. Türkçe için BM25'te Snowball stemmer kullanılır.



2.6 Reranker — Cohere Rerank-3

Özellik	Değer
Türkçe desteği	Evet (100+ dil)
Latency	~200ms
Maliyet	1000 req/ay ücretsiz, sonrası çok düşük

Self-host	Gerekmiyor
-----------	------------

2.7 Dinamik Top-K ve Score Threshold

Senaryo	search_k	threshold	rerank_k
Kısa soru-cevap	10	0.75	2
Detaylı analiz	30	0.65	5
Ses / konuşma	20	0.70	4

→ Sabit top-k değil, use-case config'i. İleride query classifier ile otomatik seçilebilir.

2.8 LLM — Agnostic

Servis LLM'i doğrudan çağırır; provider abstraction katmanı üzerinden çalışır. Her projede farklı model kullanılabilir.

Provider	Model örnekleri
openai	gpt-4o, gpt-4o-mini
anthropic	claude-sonnet-4-6, claude-haiku
gemini	gemini-2.0-flash, gemini-2.5-pro
ollama	llama3, mistral (self-hosted)

3. API Tasarımı

3.1 Endpoints

Method	Endpoint	Açıklama
POST	/ingest	Doküman ekle (async)
DELETE	/ingest/{id}	Doküman ve chunk'larını sil
GET	/ingest/{id}	Ingestion status kontrolü
POST	/query	Tek seferlik soru-cevap
POST	/chat	Conversation (history ile)
GET	/collections	Mevcut collection'ları listele
POST	/collections	Yeni collection oluştur

3.2 Authentication

Her istek header'da X-API-Key ve X-Project-ID taşır. Middleware cross-project erişimi engeller.

3.3 /ingest — Async Akış

```
POST /ingest → document_id döner (hemen)
↓
Background worker işler
↓
GET /ingest/{id} → status: pending | indexing | indexed | failed
```

4. Source Loader'lar

Source	Araç / Yöntem	Not
PDF	pymupdf (layout-aware)	Tablo ve başlıklar ayrı chunk
Web	crawl4ai	JS render destekli, nav/footer temizlenir
Audio	Whisper large-v3 + pyannote	Speaker diarization, turn bazlı chunk
Video	ffmpeg → audio → AudioLoader	Önce ses çıkarılır
DB	SQL → text serializer	Join'ler flatten edilir
Image	Gemini Vision → caption	Ayrı OCR pipeline gerekmez
Email	MIME parse	Thread → mesaj bazlı
Chat	Export parse	WhatsApp, Slack export

5. Veri Modeli

SQLite — rag_documents

```
id          TEXT PRIMARY KEY
source_type  TEXT -- pdf | web | audio | db | image | email | chat
source_ref   TEXT -- URL, dosya yolu, tablo adı
content_hash TEXT -- SHA256, re-index trigger
chunk_count  INT
status       TEXT -- pending | indexing | indexed | failed
project_id   TEXT
embedded_at  TIMESTAMP
metadata     JSON
```

SQLite — rag_chunks

```
id          TEXT PRIMARY KEY
document_id  TEXT -- rag_documents.id
qdrant_point_id TEXT -- Qdrant UUID (köprü)
chunk_index  INT
token_count  INT
created_at   TIMESTAMP
```

Qdrant — Point Payload

```
vector:      [...] -- 768 dim
payload:
  document_id: uuid -- SQLite referansı
  chunk_index: int
  text:       str  -- chunk içeriği
  source_type: str  -- filtering
  project_id: str  -- filtering
```

6. Observability

Her ingestion ve query işlemi yapılandırılmış log üretir. Debug ve maliyet takibi için kritik.

6.1 Ingestion Log

```
{
  event:      'chunk_indexed',
  document_id: 'uuid',
  chunk_index: 3,
  token_count: 287,
  embed_model: 'gemini-embedding-exp-03-07',
  embed_ms:   142,
  project_id: 'sports_app'
}
```

6.2 Query Log

```
{
  event:      'query_completed',
  query_hash: 'sha256...',
  search_k:   20,
  rerank_k:   3,
  reranker_ms: 198,
  llm_ms:     1240,
  prompt_tokens: 820,
  completion_tokens: 310,
  top_chunk_score: 0.91,
  project_id: 'sports_app'
}
```

→ Query içeriği loglanmaz - sadece hash. GDPR ve gizlilik için.

7. Ingestion Queue

Ingestion işlemleri async worker üzerinden yapılır. SQLite status polling yeterli değildir; is kuyruğu şarttır.

Secenek	Gereksinim	Oneri
ARQ	Redis, async/await native	FastAPI ile uyumlu, hafif
Celery	Redis veya RabbitMQ	Agir, overkill olabilir

RQ	Redis	Basit ama async destegi zayıf
----	-------	-------------------------------

→ ARQ secildi. Redis zaten stack'te var. Failed job'lar otomatik retry (max 3) ile yeniden kuyruğa alinir.

```
POST /ingest
|
SQLite: status=pending
|
ARQ queue'ya job ekle
|
Worker: load -> chunk -> embed -> Qdrant
|
SQLite: status=indexed (veya failed -> retry)
```

8. Rate Limiting

İki katmanlı rate limit uygulanır: API seviyesi ve embedding API seviyesi.

Katman	Limit	Kapsam
API -- /query	60 req/dk	project_id bazında
API -- /ingest	20 req/dk	project_id bazında
Embedding API	Token bucket	Global, Gemini quota'ya göre
LLM API	Token bucket	Global, provider quota'ya göre

→ Redis'te sliding window counter. Limit asılınca 429 doner, Retry-After header'i ile.

9. Embedding Versioning

Model degistiginde eski chunk'lar stale olur. Her chunk hangi model ile embed edildiği saklanmalıdır.

SQLite -- rag_chunks (guncellendi)

```
embed_model TEXT -- 'gemini-embedding-exp-03-07'
embed_version TEXT -- '1.0'
dimension INT -- 768
```

→ Model degisince stale chunk'lar tespit edilir, ARQ kuyruguna alinir, yeniden embed edilir.

10. Health Check & Circuit Breaker

10.1 Health Endpoint

```
GET /health
{
  status: 'ok' | 'degraded' | 'down',
```

```
qdrant: 'ok' | 'down',
sqlite: 'ok' | 'down',
redis: 'ok' | 'down',
embedder: 'ok' | 'rate_limited' | 'down'
}
```

10.2 Circuit Breaker Kurallari

Servis	Hata	Davranis
Qdrant	Down	Query 503, ingestion kuyruga alinir
Gemini (primary)	Rate limit	Fallback modele gec
Gemini (fallback)	Down	Ingestion durdurulur, 503
Cohere reranker	Down	Rerank atlanir, sadece RRF skoru
LLM	Down	Context doner, LLM yaniti yok, 503

11. Chunk Kalite Kontrolu

Embed oncesi her chunk kalite filtresinden gecer. Dusuk kaliteli chunk'lar index'e alinmaz.

Kural	Esik	Aksiyon
Min token	< 50 token	Bir sonraki chunk ile birlestir
Max token	> 600 token	Yeniden bol
Bos icerik	Whitespace only	Atla
Duplicate	content_hash cakismasi	Atla, mevcut chunk_id kullan
Dil tespiti	Guyen < 0.7	Logla, yine de index et

12. Re-index Stratejisi

Kismi guncelleme desteklenir -- sadece degisen chunk'lar yeniden embed edilir.

```
Yeni icerik geldi
|
content_hash hesapla
|
Hash ayni mi? -> Evet: islem yok
| Hayir
Yeni chunk'lari hesapla
|
Chunk hash'lerini karsilastir (eski vs yeni)
|
Sadece degisen chunk'lar:
```

- Qdrant'tan sil (qdrant_point_id ile)
- Yeniden embed et
- Qdrant'a ekle
- SQLite'i guncelle

→ Büyük dokumanlarda (100+ chunk) bu yaklasim embed maliyetini yuzde 60-80 azaltir.

13. LangGraph -- Opsiyonel Query Pipeline

Direkt pipeline basit soru-cevap icin yeterlidir. Self-RAG, agentic fallback veya multi-hop retrieval gerektiginde LangGraph devreye girer. Bagimlilik zorunlu degil -- config ile toggle edilir.

13.1 Ne Zaman LangGraph?

Senaryo	Direkt Pipeline	LangGraph
Basit soru-cevap	Yeterli	Gereksiz kompleks
Self-RAG (chunk kalite degerlendirme)	Manuel kod	Dogal fit
Query rewrite (yetersiz chunk)	Manuel kod	Dogal fit
Agentic fallback (web arama vs.)	Zor	Dogal fit
Multi-hop retrieval	Zor	Dogal fit

13.2 Self-RAG Akisi (LangGraph ile)

```
Query
|
[classify] -- basit mi, karmasik mi?
|
[retrieve] -- hybrid search + rerank
|
[grade] -- chunk'lar yeterli mi? (LLM ile degerlendir)
|   |
| Yeterli   Yetersiz
|   |
|   [rewrite] -- query'i yeniden yaz
|   |   max 2 tur
|   [retrieve] -- tekrar ara
|
[generate] -- LLM cevap uret
|
[hallucination_check] -- cevap context'e uygun mu?
|
Response
```

→ Hallucination check: LLM'in verdigi cevabi context ile karsilastirir, tutarsizsa yeniden generate eder.

13.3 Config Toggle


```
# /query request body
{
  'query':    '...',
  'project_id': 'sports_app',
  'config': {
    'use_graph':    true,  -- false: direkt pipeline
    'max_rewrite':  2,    -- max query rewrite turu
    'grade_threshold': 0.7, -- chunk kalite esigi
    'provider':     'openai'
  }
}
```

→ LangChain kullanılmıyor. Sadece langgraph paketi -- state machine altyapisi için.

14. Multi-Tenant İzolasyon Modeli

Servis şu an 'SaaS-ready, single-operator' modunda çalışır. Kendi projelerin farklı tenant olarak tanımlanır. Mimaride tenant_id her yerde mevcuttur -- ileride public SaaS açıldığında sadece kayıt ve billing katmanı eklenir, core'a dokunulmaz.

14.1 Uc Katmanlı İzolasyon

Katman	Alan	Açıklama
1	tenant_id	En üst seviye. CRM'de müşteri firma, mobil app'te uygulamanın kendisi.
2	scope_type	tenant user tenant_user -- izolasyon seviyesi
3	scope_id	tenant_id altındaki kimlik. User ID, tenant kodu veya ikisinin birleşimi.

14.2 Proje Tiplerine Göre Konfigurasyonlar

Proje Tipi	tenant_id	scope_type	scope_id	Açıklama
Mobil app (sports)	sports_app	user	user_456	Her kullanıcı kendi verisini görür
CRM -- firma bazlı	crm	tenant	tenant_abc	Firma çalışanları veriyi paylaşar
CRM -- kullanıcı bazlı	crm	tenant_user	tenant_abc:user_789	Her çalışanın özel verisi var
Tekil proje	personal_tool	tenant	null	Sadece sen kullanırsın

→ CRM'de iki mod birlikte çalışabilir: firma geneli shared index (scope_type=tenant) + çalışanlara özel index (scope_type=tenant_user). Query'de hangi scope sorgulanacağı belirtilir.

14.3 Qdrant Payload & Filter

```
# Her chunk'a eklenen payload
{
  'tenant_id': 'crm',
  'scope_type': 'tenant_user',
  'scope_id': 'tenant_abc:user_789',
  'text':     '...',
}
```

```
'source_type': 'pdf'
}

# Query middleware'i otomatik inject eder
filter = {
  'tenant_id': 'crm',
  'scope_id': 'tenant_abc:user_789'
}
```

→ Kullanici baska tenant veya kullanicinin verisine hicbir sekilde erisemez. Filter servis tarafinda inject edilir, client'tan gelmez.

15. SaaS Yolculugu -- Adim Adim

Simdi: kendi projelerini calistiran, SaaS-ready altyapi. Gelecekte: public SaaS. Her adim bir oncekinin uzerine infaat eder, yikim yoktur.

Asama 1 -- Simdi: Single-Operator (Mevcut)

Yapilan	Detay
Multi-tenant mimari	tenant_id + scope_type + scope_id her yerde mevcut
API key yonetimi	Her proje icin ayri key, Laravel'de manuel tanimlanir
Izolasyon	Qdrant payload filter, SQLite tenant_id kolonu
Rate limiting	Redis, tenant bazli -- aktif
Usage metering	Redis counter -- aktif ama billing yok, sadece observability
Billing	Yok -- kendi projelerinde gereksiz
Public kayit	Yok

Asama 2 -- Yakin: Internal Multi-Project Olgunlasma

Kendi projelerinde production'a gecince bu adimlar tamamlanir:

Adim Is	Neden
2.1 Filament admin panel -- tenant listesi, API key yonetimi	Mayonezi yukunu kaldir
2.2 Usage dashboard -- proje bazli token/query limitleri	Her projeye kadar harcadi gorursun
2.3 Webhook callback sistemi -- ingestion tamamlandicta tetikler	DevOps'te kolaylasir
2.4 SDK -- Laravel HTTP wrapper sinifi	Her projede tekrar yazmamak icin
2.5 Staging ortami -- ayri Qdrant + SQLite instance'leri	Production'a zarar vermeden test

Asama 3 -- SaaS Beta: Ilk Dis Kullanicilar

Adim Is	Onkosul
---------	---------

3.1	Self-serve kayıt -- email + sifre, API key otomatik olarak oluşturulmalı	Asama 2 tamamlanmış olmalı
3.2	Plan limitleri -- Free tier acilir, hard limit enforsement, usage metering hazır olmalı	Asama 2 tamamlanmış olmalı
3.3	Stripe entegrasyonu -- Starter + Growth planları için fiyatlandırma netlesmiş olmalı	Asama 2 tamamlanmış olmalı
3.4	Public dokümanlar -- docs sitesi, quick start SOW hazır olmalı	Asama 2 tamamlanmış olmalı
3.5	Email bildirimleri -- limit uyarısı, ingestion hatası	Herhangi bir zamanda eklenebilir

Asama 4 -- SaaS GA: Tam Ürün

Adım	İs
4.1	Business plan -- SSO/SAML, SLA, Slack support kanalı
4.2	Enterprise -- dedicated Qdrant collection veya self-hosted seçenek
4.3	Marketplace entegrasyonları -- n8n, Zapier, Make.com node
4.4	Retrieval quality analytics -- hangi chunk kaç kere dondu, ortalama skor
4.5	Vertical landing pages -- Legal, E-commerce, Call Center

→ Her asama bağımsızdır. Asama 3'e geçmek için Asama 4'u beklemeye gerek yok. Her adım incremental gelir sağlar.

16. Asama 1 Gelistirmeleri

Mevcut plan üzerine eklenecek özellikler. Temel servis tamamlandıktan sonra bu geliştirmeler sırasıyla eklenir. Her biri bağımsız, birbirini bozmaz.

16.1 Ingestion Webhook Callback

Su an polling var. Laravel 'bitti mi?' diye sormak yerine 'bitti, su URL'e haber ver' diyebilmeli.

```
# /ingest request'e ekle
{
  'source_type': 'pdf',
  'callback_url': 'https://myapp.com/webhooks/rag',
  'callback_secret': 'whsec_xxx'
}

# Ingestion tamamlanınca sistem çağırır
POST callback_url
X-RAG-Signature: sha256=...
{
  'document_id': 'uuid',
  'status': 'indexed',
  'chunk_count': 42
}
```

→ Tahmini süre: 0.5 gün. Laravel entegrasyonu anında kolaylaşır.

16.2 Query Cache

Aynı soru tekrar gelirse Qdrant'a gitmeden Redis'ten cevap don. Maliyet ve latency düşer.

```
cache_key = sha256(query + tenant_id + scope_id + collection)
```

Redis'te var mı? -> Evet: direkt don (0ms)

Hayır: pipeline çalıştır, sonucu cache'e yaz

TTL: 1 saat (konfigurasyon ile değiştirilebilir)

Cache invalidation: ilgili collection re-index edilince

→ Tahmini süre: 1 gün. Tekrarlayan sorgularda maliyet %40-60 düşürülebilir.

16.3 SummaryFormatter

Ham veriyi embed etmeden önce natural language'a çevir. DB kayıtları, JSON, tablo verisi -- bunları direkt embed etmek kaliteyi düşürür.

```
# Öncesi (ham, embedding kalitesi düşük)
{price: 150, stock: 0, category: 'elektronik', sku: 'ABC123'}
```

```
# Sonrası (natural language, embedding kalitesi yüksek)
'Elektronik kategorisinde ABC123 kodlu ürün.
Fiyatı 150 TL. Şu an stokta bulunmuyor.'
```

Source	Formatter
DB record	Alan adı + değer kombinasyonu, JOIN'ler açıklanmış
JSON payload	Key'ler Türkçe/İngilizce label'a dönüştürülür
Tablo / Excel	Satır bazlı cümle üretimi
Form verisi	Alan adı + değer, boş alanlar atlanır

→ Tahmini süre: 1-2 gün. Özellikle DB ve JSON ingestion kalitesini ciddi artırır.

16.4 Query Expansion

Kullanıcı 'fiyat' dedi, sistem 'ücret, maliyet, tutar, bedel' ile de arar. Türkçe morfoloji için kritik -- farklı kelimelerle ifade edilen aynı kavramları yakalar.

```
# Yöntem A: Sinonim sözlüğü (hızlı, deterministik)
expansions = {
  'fiyat': ['ücret', 'maliyet', 'tutar', 'bedel', 'para'],
  'iptal': ['vazgeçme', 'geri alma', 'iptal etme'],
  'teslimat': ['kargo', 'gönderim', 'ulaştırma']
}
```

```
# Yöntem B: LLM ile dinamik genişleme (daha akıllı)
expanded = llm.expand_query('fiyat nedir?')
# -> ['fiyat nedir', 'ürün maliyeti', 'ne kadar tutar', 'ücret bilgisi']
```

→ Tahmini süre: 1 gün. Türkçe RAG'da retrieval precision'i belirgin artırır.

16.5 Conversation Memory (Redis Session)

Su an /chat endpoint'inde history client'tan geliyor. Session bazli hafiza Redis'te tutulsa client her seferinde history gondermek zorunda kalmaz.

```
# İlk istek
POST /chat
{'query': 'iade politikasi nedir?', 'tenant_id': 'crm'}
Response: {'answer': '...', 'session_id': 'sess_abc'}

# Takip sorusu -- history gondermek zorunda degil
POST /chat
{'query': 'peki kargo ucreti kim oder?', 'session_id': 'sess_abc'}
# Sistem 'peki' nin onceki soruya referans oldugunu anlar
```

→ Session TTL: 30 dakika. Son 6 tur saklanir (token israfi onlemek icin). Tahmini sure: 1 gun.

16.6 Batch Ingest

Tek tek degil, toplu dokuman gonderilebilmeli. 100 PDF'i tek API cagrisiyla kuyruğa almak.

```
POST /ingest/batch
{
  'documents': [
    {'source_type': 'pdf', 'source_ref': 'base64...', 'metadata': {}},
    {'source_type': 'web', 'source_ref': 'https://...'},
    {'source_type': 'db', 'source_ref': 'table:products'}
  ],
  'tenant_id': 'crm',
  'callback_url': 'https://myapp.com/webhooks/rag'
}

Response:
{
  'batch_id': 'batch_uuid',
  'document_ids': ['uuid1', 'uuid2', 'uuid3'],
  'status': 'queued'
}
```

→ Tahmini sure: 1 gun. İlk kurulum ve migration senaryolari icin sart.

16.7 Scheduled Re-index

Web kaynaklari degisir. 'Bu URL'i her gece yeniden indexle' gibi cron bazli ingestion.

```
POST /schedules
{
  'document_id': 'uuid',
  'cron': '0 2 * * *', -- her gece 02:00
  'enabled': true
}

# SQLite'a yeni tablo
rag_schedules:
  document_id, cron, last_run, next_run, enabled
```

→ Tahmini sure: 1 gun. Web kaynaklari ve surekli degisen DB verileri icin kritik.

16.8 Confidence Score

Cevapla birlikte 'bu cevaba ne kadar guveniyorum' skoru don. Dusuk confidence'ta uygulama uyarı gosterebilir.

```
# Query response'a eklenir
{
  'answer': '...',
  'confidence': 0.87,    -- top chunk score ortalamasından türetilir
  'sources': [...],
  'warning': null      -- confidence < 0.6 ise 'Dusuk guven' uyarısı
}
```

→ Tahmini süre: 0.5 gün. Hallucination riskini kullanıcıya bildirmek için önemli.

16.9 Negative Filtering

Belirli dokümanları veya source type'leri query'den hariç tut.

```
POST /query
{
  'query': '...',
  'config': {
    'exclude_sources': ['web'],    -- web kaynaklarını atla
    'exclude_documents': ['uuid1', 'uuid2'] -- bu dokümanları atla
  }
}
```

→ Tahmini süre: 1 gün. Hassas dokümanları belirli sorgulardan gizlemek için kullanılır.

16.10 Document Relationship

Chunk'lar birbirinden habersiz. Uzun dokümanlarda bölümler arası referanslar kaybolur. Metadata'da ilişkili chunk'lar işaretlenir.

```
# rag_chunks tablosuna eklenir
related_chunks JSON -- ['chunk_uuid_1', 'chunk_uuid_2']
parent_section TEXT -- 'Bölüm 3: Ödeme Koşulları'

# Retrieval sırasında ilişkili chunk'lar da getirilir
# 'Bu chunk getirildi' -> ilişkili chunk'lar da context'e eklenir
```

→ Tahmini süre: 2-3 gün. Uzun hukuki/teknik dokümanlarda cevap kalitesini artırır.

16.11 Öncelik Sirası

Oncelik	Ozellik	Etki	Tahmini Sure
1	Ingestion webhook callback	Laravel entegrasyonu kolaylasir	0.5 gun
2	Query cache (Redis)	Maliyet + latency duser	1 gun
3	SummaryFormatter	Embedding kalitesi artar	1-2 gun
4	Query expansion	Turkce retrieval kalitesi artar	1 gun
5	Conversation memory	Chat deneyimi duzlenir	1 gun
6	Batch ingest	Toplu veri icin sart	1 gun
7	Scheduled re-index	Web/DB kaynaklari icin sart	1 gun

8	Confidence score	Hallucination kontrolu	0.5 gun
9	Negative filtering	Guclu query kontrolu	1 gun
10	Document relationship	Uzun dokuman kalitesi	2-3 gun

17. Kullanım Stratejisi — Önce Kendi Projeler, Sonra SaaS

Servis ilk etapta sadece kendi projelerinde (CRM, sports app) kullanılacak. UX, dashboard veya dış kullanıcı yok. Odak: doğru cevap, düşük maliyet, debug edilebilirlik. SaaS kapısı açık tutulur ama şimdi implement edilmez.

→ Bu perspektif önceliklendirmeyi doğrudan etkiler. UX gerektiren özellikler (streaming, widget, onboarding) SaaS aşamasına ertelenir.

17.1 Büyük Dosya Stratejisi — URL vs Base64

PDF, video, audio gibi büyük dosyalar Laravel'den Python servisine iki yöntemle gönderilebilir. 50MB üstü dosyalar için URL yöntemi zorunludur.

Yöntem	Limit	Kullanım	Not
Base64 (body)	<= 50MB	Küçük PDF, kısa ses	Memory + network pahalisi
URL (S3/Hetzner)	Sınır yok	Video, büyük audio, bulk PDF	Tercih edilen yöntem

```
# Tercih edilen akış (URL ile)
1. Laravel -> dosyayı Hetzner Object Storage'a yükle
2. Laravel -> RAG servisine sadece URL gönder
3. Python -> URL'den indir, işle, embed et, Qdrant'a yükle
4. Python -> callback_url'e bildir (status: indexed)
```

```
# Küçük dosyalar için (base64)
POST /ingest
{
  'source_type': 'pdf',
  'source_ref': 'base64:JVBERi0xLjQK...',
  'tenant_id': 'crm'
}
```

18. Geliştirme Yol Haritası — Gruplu Öncelikler

Özellikler üç gruba ayrılmıştır: servis çalışsın diye core'a eklenmesi gerekenler, production'da ihtiyaç duyulacaklar ve SaaS'a geçince gerekecekler.

18.1 Grup A — Core (Servis Tamamlanmadan Eklenmeli)

Mimari kararları etkileyen özellikler. Sonradan eklemek breaking change veya veri migrasyonu gerektirir.

#	Ozellik	Neden Core	Sure
A1	Sync / Async ingest modu	ingest?mode=sync async -- kucuk ingestion'larda Async overkill	0.5 gun
A2	Multi-collection query	API tasarimini etkiler, sonradan eklemek breaking change olur	1 gun
A3	Worker ayrimi (CPU vs IO)	Whisper (CPU-bound) diger IO islerini bloklar, ayri pool lazim	1 gun
A4	Chunk seviyesi ACL	Qdrant payload'a simdi eklenmezse tum chunk'lar yeniden index gerekir	1 gun
A5	Buyuk dosya: URL desteği	Video/audio base64 ile gonderilemez, URL pipeline'osind	5 guntasarlanmali

```
# A2: Multi-collection query ornegi
POST /query
{
  'query': 'iade politikasi nedir?',
  'collections': ['politikalar', 'urunler', 'musteri_gecmisi'],
  'merge_strategy': 'rrf' -- weighted de olabilir
}

# A3: Worker ayrimi
arq_worker_io -- embedding, web crawl, DB sorgu (IO-bound)
arq_worker_cpu -- Whisper, ffmpeg, pyannote (CPU-bound)

# A4: Chunk ACL payload ornegi
{
  'tenant_id': 'crm',
  'acl': ['admin', 'manager'], -- bos liste = herkese acik
  'text': '...'
}
```

18.2 Grup B — Production (Ilk Gercek Proje Kullanimi Sonrasi)

Production'a girince ihtiyac duyulacak ozellikler. Ilk proje (sports app veya CRM) kullanima alinince bunlar devreye girer.

#	Ozellik	Tetikleyici	Sure
B1	Semantic deduplication	Ayni icerik farkli formatlarda gelince index sise	2 gun
B2	RAG Evaluation Pipeline	"Neden yanlis cevap verdi?" sorusu gelmeden on	1 gun
B3	SQLite -> PostgreSQL plani	10K+ dokumanda write lock sorunu baslar	1 gun (plan)
B4	Adaptive chunking	Icerik cesitliligi artinca sabit chunk size yetersiz	2 gun
B5	Feedback loop	Gercek kullanici verisi biriktikce sistemi iyilestir	2 gunc

```
# B1: Semantic deduplication
# Content hash ayni formattaki duplicate'leri yakalar
# Farkli format, ayni icerik icin embedding similarity kontrolu
similarity = cosine(new_embedding, existing_embeddings)
if similarity > 0.97: # threshold
    skip_indexing() # duplicate, atlat

# B2: RAG Evaluation -- RAGAS metrikleri
```



```

metrics = [
  'faithfulness',    # cevap context'e uygun mu?
  'answer_relevancy', # cevap soruya uygun mu?
  'context_recall',  # dogru chunk'lar getirildi mi?
]

# B5: Feedback loop
POST /feedback
{
  'query_id': 'uuid',
  'rating': -1,    -- -1 kötü, +1 iyi
  'chunk_ids': ['uuid1'] -- hangi chunk'lar kotuydu
}

```

18.3 Grup C — SaaS (Dis Kullanici Geline)

Sadece kendi projelerinde kullanirken gerek yok. İlk dis kullanıcı veya SaaS beta'ya gecince devreye girer.

#	Ozellik	Tetikleyici	Sure
C1	Streaming response	Dis kullanıcı UX beklentisi -- token token goster	1 gun
C2	Multimodal query	E-ticaret musterisi: "bu gorseldeki urunu bul"	2 gun
C3	Hot/cold storage	Qdrant maliyeti artinca eski chunk'lari arsivle	2 gun
C4	Self-serve kayıt	SaaS beta -- email + sifre, API key otomatik	Asama 3
C5	Stripe entegrasyonu	Ilk odeme alinacagi zaman	Asama 3

```

# C1: Streaming -- FastAPI SSE
POST /query?stream=true
Content-Type: text/event-stream
data: {"token": "Merhaba"}
data: {"token": " iade"}
data: {"token": " politikasi..."}
data: [DONE]

# C2: Multimodal query
POST /query
{
  'query_type': 'image',
  'image':      'base64...',
  'text':      'bu urunu bul' -- opsiyonel
}

```

18.4 Yol Haritasi Ozeti

Grup	Ne Zaman	Toplam Sure	Ozellikler
A — Core	Servis yazilirken	~4 gun	Sync/async, multi-col, worker ayrimi, ACL, URL upload

B — Production	Ilk proje production'a girince	~10 gun	Semantic dedup, RAG eval, SQLite plan, adaptive chunk, fe
C — SaaS	Ilk dis kullanıcı gelince	~7 gun+	Streaming, multimodal, hot/cold, kayıt, ödeme

→ Grup C özelliklerini şimdi implement etme. Sadece mimariyi kirmasin diye akilda tut.

19. LLM Council Bulguları — Eksik Kararlar

RAG service planı LLM Council (llmcouncil.ai) ile değerlendirildi. Aşağıdaki eksikler tespit edildi ve plana eklendi. Kaynagi: Production-Ready Postgres DDL, Partitioning Strategy & LangGraph Orchestration for Multi-Tenant RAG.

19.1 Document Versioning

Doküman değişince sadece hash güncellemek yetmez. Her değişiklik yeni bir version oluşturmali, eski version korunmalı. Rollback ve audit için kritik.

```
-- rag_documents tablosuna eklenir
version          INT DEFAULT 1,
previous_document_id UUID -- hangi versiyondan guncellendi

-- Akis
Yeni doküman geldi -> content_hash degisti mi?
Hayir -> last_sync_at guncelle, islem yok
Evet -> version + 1, previous_document_id = eski id
-> yeni ingestion job baslar
```

→ Planin önceki versiyonunda sadece content_hash vardı. Version + previous_document_id Core'a eklendi.

19.2 Chunk Soft-Delete

Chunk silinmemeli, arşivlenmeli. Qdrant'tan kaldırılır ama SQLite'ta is_archived=True olarak tutulur. Diff log ve maliyet analizi için gerekli.

```
-- rag_chunks tablosuna eklenir
is_archived      BOOLEAN DEFAULT FALSE,
status: 'pending' | 'embedded' | 'failed' | 'archived'

-- Silme akisi
Chunk artık dokümanda yok
-> SQLite: is_archived = TRUE, status = 'archived'
-> Qdrant: qdrant_point_id ile sil
-> diff_log: operation = 'deleted'

-- Sorgularda her zaman filtrele
WHERE is_archived = FALSE
```

19.3 rag_chunk_diff_log Tablosu

Her ingestion'da hangi chunk'ların new/modified/deleted/unchanged olduğu loglanır. Debug, maliyet analizi ve audit için çok değerli.

```
-- Yeni tablo: rag_chunk_diff_log
diff_id      UUID PRIMARY KEY
tenant_id    UUID
document_id  UUID
job_id       UUID -- hangi ingestion job'u
operation     TEXT -- 'new' | 'modified' | 'deleted' | 'unchanged'
chunk_id     UUID
old_content_hash CHAR(64)
new_content_hash CHAR(64)
created_at   TIMESTAMPTZ
```

→ Her ingestion sonunda kac chunk new/modified/unchanged/deleted oldugu gorulur. Embed maliyet optimizasyonu icin kritik veri.

19.4 rag_ingestion_jobs Tablosu

ARQ job'larini takip etmek yetmez. Her ingestion job'u veritabaninda kayit altinda olmalı. Retry sayisi, sure, basarili/basarisiz chunk sayisi izlenir.

```
-- Yeni tablo: rag_ingestion_jobs
job_id       UUID PRIMARY KEY
tenant_id    UUID
project_id   UUID
document_id  UUID
job_type     TEXT -- 'initial_ingest' | 'sync' | 'reindex'
status       TEXT -- 'queued' | 'running' | 'completed' | 'failed'
retry_count  INT DEFAULT 0
max_retries  INT DEFAULT 3
started_at   TIMESTAMPTZ
completed_at TIMESTAMPTZ
duration_ms  INT
chunks_processed INT
chunks_failed INT
error_message TEXT
created_at   TIMESTAMPTZ
```

19.5 rag_sync_checkpoints Tablosu

Scheduled re-index icin connector bazli sync durumu. 'Bu web kaynakini son nereye kadar cektim?' bilgisi olmadan incremental sync mumkun degil.

```
-- Yeni tablo: rag_sync_checkpoints
checkpoint_id UUID PRIMARY KEY
tenant_id     UUID
connector_id  TEXT -- kaynak identifier
connector_type TEXT -- 'web' | 'db' | 'email' | 's3'
cursor_state  TEXT -- son sync noktasi (URL, timestamp, offset)
last_sync_time TIMESTAMPTZ
status        TEXT -- 'active' | 'paused' | 'error'
consecutive_failures INT DEFAULT 0
created_at    TIMESTAMPTZ
updated_at    TIMESTAMPTZ

-- Ornek cursor_state degerler
Web -> son crawl edilen URL veya timestamp
```

DB -> son islenen row ID veya updated_at
Email -> son islenen message ID

19.6 Latency + Token Budget Enforcement

LangGraph pipeline'ında her query için maksimum süre ve token limiti tanımlanmalı. Limit asılınca erken abort yapılır, partial cevap veya hata doner.

```
# QueryState'e eklenir
latency_budget_ms: int # ornek: 3000ms
token_budget:      int # ornek: 2000 token
tokens_used:       int # anlık takip
start_time:        float # pipeline baslangici

# Her node'da kontrol
elapsed = (time.time() - state['start_time']) * 1000
if elapsed > state['latency_budget_ms']:
    return early_abort(state) # partial veya cached cevap don

# Query config'den gelebilir
POST /query
{
  'query': '...',
  'config': {
    'latency_budget_ms': 3000,
    'token_budget':     2000
  }
}
```

→ Maliyet kontrolü ve predictable latency için kritik. Özellikle self-RAG rewrite döngüleri bu olmadan kontrolden çıkabilir.

19.7 bbox Chunk Metadata

PDF chunk'larının sayfa üzerindeki koordinatları. Vision-RAG ve görsel referans için gerekli. Şimdi eklenmezse tüm PDF'ler yeniden index gerektirir.

```
-- rag_chunks tablosuna eklenir
page_number INT -- hangi sayfada
bbox        JSONB -- {x1, y1, x2, y2} piksel koordinatları
section_title TEXT -- 'Bölüm 3: Ödeme Koşulları'

-- Kullanım
# Cevap verilirken kaynak göster
{'answer': '...', 'source': {'page': 7, 'bbox': {x1:100, y1:200, ...}}}
```

→ source_connector_id da eklendi: hangi connector'dan (web/db/s3) geldi bilgisi. Scheduled re-index ve sync checkpoint ile birleşince güçlü.

19.8 PgBouncer (PostgreSQL Gecisinde)

PostgreSQL'e geçildiğinde connection pooling zorunludur. FastAPI + ARQ worker'lar çok sayıda concurrent connection açar, PgBouncer olmadan PostgreSQL bunalır.

Parametre	Deger	Aciklama
pool_mode	transaction	Her transaction sonrasi connection serbest bırakılır
pool_size	20-50	Toplam worker sayısına gore ayarlanır
max_client_conn	200	Uygulama tarafindaki max baglanti
server_idle_timeout	600s	Bos connection kapat

→ Simdi implement etme. SQLite'tan PostgreSQL'e gececegin zaman PgBouncer'i de Docker Compose'a ekle.

19.9 Guncellenmis Veri Modeli

Yukaridaki bulgular ile SQLite sema guncellendi. PostgreSQL'e gecildiginde bu sema dogrudan kullanilir.

```
-- rag_documents (guncellendi)
id          TEXT PRIMARY KEY
source_type TEXT
source_ref  TEXT
source_connector_id TEXT -- YENİ
content_hash CHAR(64)
file_size_bytes BIGINT -- YENİ
file_type    TEXT -- YENİ
title        TEXT -- YENİ
version      INT DEFAULT 1 -- YENİ
previous_document_id TEXT -- YENİ
status       TEXT
chunk_count  INT
project_id   TEXT
tenant_id    TEXT
last_sync_at TIMESTAMP -- YENİ
embedded_at  TIMESTAMP
metadata     JSON

-- rag_chunks (guncellendi)
id          TEXT PRIMARY KEY
document_id TEXT
qdrant_point_id TEXT
chunk_index INT
token_count INT
embed_model TEXT
embed_version TEXT
dimension   INT
page_number INT -- YENİ
bbox        JSON -- YENİ {x1,y1,x2,y2}
section_title TEXT -- YENİ
acl         JSON -- YENİ ['admin','manager']
is_archived BOOL DEFAULT FALSE -- YENİ
status      TEXT -- YENİ: pending|embedded|failed|archived
version     INT DEFAULT 1 -- YENİ
related_chunks JSON -- mevcut
created_at  TIMESTAMP
updated_at  TIMESTAMP -- YENİ
```

```
-- YENİ TABLOLAR
rag_ingestion_jobs    -- job takibi
rag_chunk_diff_log    -- delta audit
rag_sync_checkpoints  -- connector sync durumu
```

20. Deployment (Sonraya Birakildi)

Deployment kararları (Docker Compose yapısı, Nginx config, Hetzner VPS yerleşimi, Coolify/Dokploy entegrasyonu) ayrı bir belgede ele alınacaktır.

21. Karar Özeti

Karar Alanı	Secim	Gerekçe
Vector Store	Qdrant (Docker)	Guclu filtering, native hybrid, izole
Registry	SQLite	Sifir ops, servis ici, tasinabilir
Embedding Primary	gemini-embedding-exp-03-07 @ 768	Turkce guclu, multimodal
Embedding Fallback	text-embedding-004 @ 768	Stabil, rate limit yok
Dimension	768 (sabit, truncated)	Collection degismez, model bagimsiz
Chunking	Source-aware + parent-child	Bagiam korunur, maliyet dengeli
Search	Hybrid (Dense + BM25 + RRF)	Turkce'de keyword+semantic sart
Reranker	Cohere Rerank-3	Turkce, dusuk latency, ucuz
LLM	Agnostic (provider abstraction)	Her proje farkli model secebilir
Top-K	Dinamik + score threshold	Use-case'e gore ayarlanabilir
Queue	ARQ + Redis	Async, retry, FastAPI uyumlu
Rate Limit	Redis sliding window	project_id bazinda, 429 + Retry-After
Versioning	embed_model kolonu	Stale chunk tespiti, kismi re-index
Health	/health endpoint	Her bagimlinin durumu ayri izlenir
Circuit Breaker	Per-service kurallar	Graceful degradation, fallback
Chunk Kalite	Min/max token + duplicate filter	Dusuk kaliteli chunk index'e girmez
Re-index	Chunk-level hash karsilastirma	Sadece degisen chunk'lar embed edilir
Observability	Structured JSON log	Token maliyet + latency takibi
Query Pipeline	Direkt + LangGraph (opsiyonel)	Self-RAG, rewrite, hallucination check
Tenant modeli	tenant_id + scope_type + scope_id	Iskencilik, multi-user, multi-tenant destegi

SaaS yolculugu	4 asamali incremental	Core degismez, katman katman SaaS eklenir
Webhook callback	HMAC imzali async bildirim	Polling yerine event-driven entegrasyon
Query cache	Redis TTL cache	Maliyet + latency optimizasyonu
SummaryFormatter	Source-aware natural language	Ham verinin embedding kalitesini arttirir
Query expansion	Sinonim + LLM genisleme	Turkce retrieval precision artisti
Session memory	Redis session (30dk TTL)	Stateful chat, history gondermek gerekmez
Batch ingest	POST /ingest/batch	Toplu dokuman yukleme
Scheduled re-index	Cron bazli ARQ job	Web/DB kaynaklari guncel kalir
Confidence score	Top chunk score ortalamasindan	Dusuk guven uyarisi
Negative filtering	exclude_sources + exclude_documents	Yanlis dokuman kontrolu
Document relationship	related_chunks metadata	Bolumler arasi referans korunur
Worker tipi	CPU pool + IO pool (ayri)	Whisper diger isler bloklamasin
Chunk ACL	acl[] payload alani	Chunk seviyesi erisim kontrolu, simdi ekle
Multi-collection	collections[] + merge_strategy	API breaking change olmadan sonra eklenemez
Ingest modu	sync async toggle	Kucuk ingestion'larda async gereksiz
Buyuk dosya	URL (Hetzner Object Storage)	50MB ustu icin URL pipeline zorunlu
Semantic dedup	Embedding similarity > 0.97	Farkli format, ayni icerik tespiti (Prod)
RAG Evaluation	RAGAS metrikleri	Retrieval kalite olcumu (Prod)
SQLite scaling	PostgreSQL gecis plani	10K+ dokumanda kilit sorunu (Prod)
Streaming	FastAPI SSE	Dis kullanici UX -- SaaS asamasinda
Multimodal query	Gemini Vision embed	Gorsel ile sorgulama -- SaaS asamasinda
Hot/cold storage	Qdrant arshivleme	Maliyet optimizasyonu -- SaaS asamasinda
Kullanim stratejisi	Once kendi projeler, sonra SaaS	UX ozellikleri ertelendi, core odak
Doc versioning	version + previous_document_id	Rollback ve audit -- Council bulgusu
Chunk soft-delete	is_archived + status=archived	Diff log icin Qdrant'tan kaldir, SQLite'ta tut
Diff log	rag_chunk_diff_log tablosu	Her ingestion delta audit -- Council bulgusu
Ingestion jobs	rag_ingestion_jobs tablosu	ARQ disinda DB seviyesi job takibi
Sync checkpoint	rag_sync_checkpoints tablosu	Connector bazli incremental sync durumu
Budget enforcement	latency_budget_ms + token_budget	LangGraph early abort, maliyet kontrolu
bbox metadata	page_number + bbox JSONB	PDF koordinat, Vision-RAG hazirligi
Connector ID	source_connector_id alani	Hangi connector kaynak, sync ile birlesir
PgBouncer	transaction pool_mode, 20-50	PostgreSQL gecisinde connection pooling

Partition stratejisi	Tiered: shared<100K, dedicated>=100K tenant otomatik dedicated partition
----------------------	--